

PBD_Assignment_3

2025-11-29

Name1: Adwitiya Argha Priyadarshini Boruah
Matriculation No. 1: 7070291
Email1: adbo00002@stud.uni-saarland.de

Name2: Md Mobashir Rahman
Matriculation No. 2: 7059086
Email2: mdra00001@stud.uni-saarland.de

Exercise 3.1: Denoising of images using a diffusion filter

a. The noisy matrix from “noise.csv” is loaded and used as the initial state. The diffusion parameters follow the assignment specification: spatial step $h=1$, time step $\Delta t=0.1$, and diffusion constant $c = 1$, resulting in $k = 0.1$. A diffusion function updates each interior pixel based on its four neighbouring values, while boundary values remain fixed. The function is applied for 10, 30, and 50 time steps to examine how the degree of smoothing changes. All results are plotted together with the original matrix for direct comparison.

The diffusion equation serves as a denoising method because it reduces high-frequency noise by averaging neighbouring intensities. Increasing the number of time steps strengthens this smoothing effect, making it possible to study how diffusion duration influences noise reduction and structural preservation.

Loading the noisy density matrix from the CSV file

```
raw_data <- as.matrix(read.csv("noise.csv", header = FALSE))
```

Making a copy that we will update during diffusion

```
initial_matrix <- raw_data
```

#Defining diffusion-related parameters

```
space_step <- 1          # h is the distance between grid points.
```

```
time_step <- 0.1         # Δt as required in the assignment
```

```
diffusion_const <- 1     # c is chosen to be 1 as commonly used
```

*# Computing $k = c * \Delta t / h^2$*

```
k_value <- diffusion_const * time_step / (space_step^2)
```

Saving number of rows and columns for later use

```
num_rows <- nrow(initial_matrix)
```

```
num_cols <- ncol(initial_matrix)
```

3. Creating a function that performs diffusion for a given number of time steps. This version keeps b

```
diffuse_matrix <- function(input_matrix, steps, k) {
```

Working on a copy so original matrix stays unchanged

```
  updated_matrix <- input_matrix
```

```

# Looping over each timestep
for (t in 1:steps) {

  # This is a temporary matrix to store new values
  next_matrix <- updated_matrix

  # Updating the interior points only and the borders remain fixed
  for (row in 2:(num_rows - 1)) {
    for (col in 2:(num_cols - 1)) {

      # Applying the discrete diffusion equation
      next_matrix[row, col] <- k * (
        updated_matrix[row + 1, col] +
        updated_matrix[row - 1, col] +
        updated_matrix[row, col + 1] +
        updated_matrix[row, col - 1]
      ) + (1 - 4 * k) * updated_matrix[row, col]

    }
  }

  # Updating the matrix after finishing this timestep
  updated_matrix <- next_matrix
}

return(updated_matrix)
}

# 4. Running the diffusion for different step counts to see how smoothing behaves.

smoothed_10 <- diffuse_matrix(initial_matrix, 10, k_value)
smoothed_30 <- diffuse_matrix(initial_matrix, 30, k_value)
smoothed_50 <- diffuse_matrix(initial_matrix, 50, k_value)

# 5. Plotting the matrices for comparison: original vs smoothed with different numbers of diffusion st

par(mar = c(4, 4, 2, 1)) #normal margins
par(mfrow = c(2, 2)) # Showing four plots in a 2x2 grid

image(initial_matrix,
      col = gray.colors(256),
      main = "Original (Noisy Matrix)")

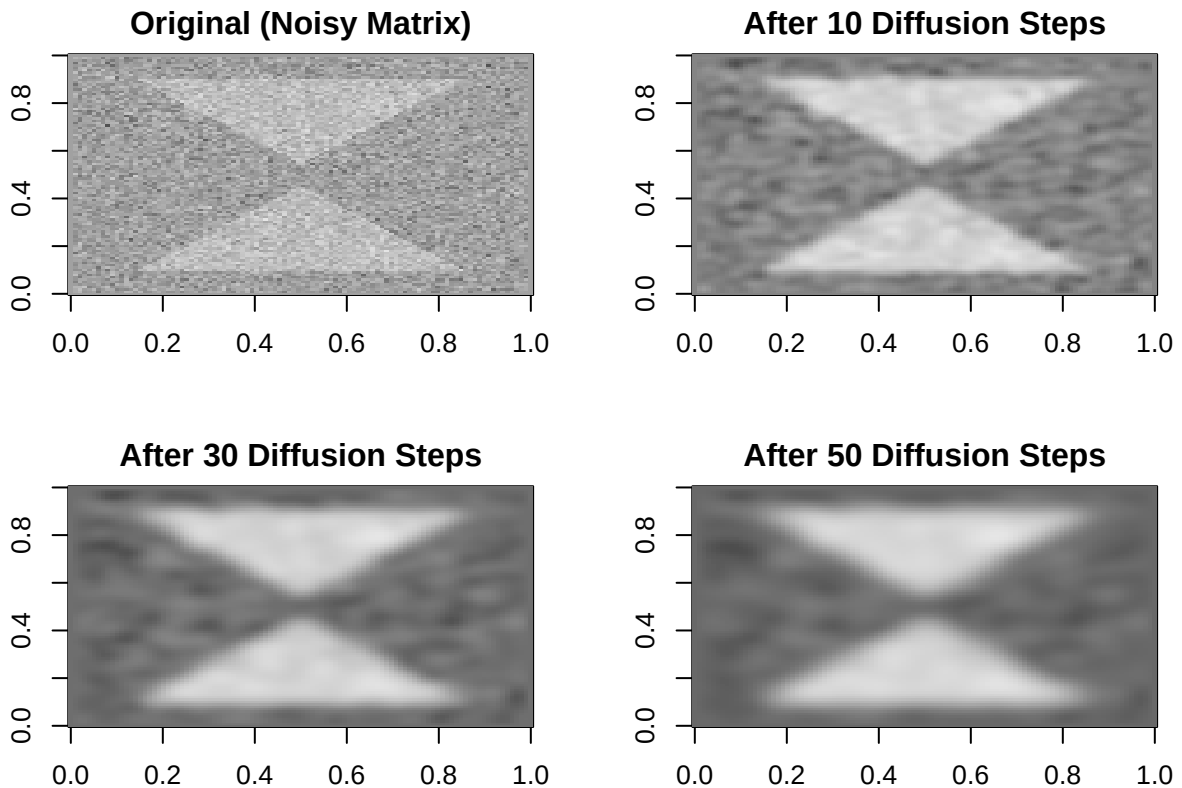
image(smoothed_10,
      col = gray.colors(256),
      main = "After 10 Diffusion Steps")

image(smoothed_30,
      col = gray.colors(256),
      main = "After 30 Diffusion Steps")

image(smoothed_50,
      col = gray.colors(256),

```

```
main = "After 50 Diffusion Steps")
```



The original matrix shows strong pixel-level noise that obscures the underlying pattern. After 10 steps, the noise decreases slightly but remains clearly visible. After 30 steps, the noise is substantially reduced, revealing the triangular structure while maintaining its definition. After 50 steps, the matrix becomes over-smoothed, noise is reduced further, but important structural boundaries blur and overall sharpness is lost. These results demonstrate the typical behaviour of the diffusion equation, and the additional time steps increase smoothing but eventually remove essential details. A moderate diffusion duration of approximately 30 steps provides the best balance between denoising effectiveness and preservation of structure for this dataset.

b. The amount of smoothing produced by each diffusion step depends directly on the size of the time step, because $k=\Delta t$ for the given parameters. A larger Δt means that each iteration performs a stronger update. To keep the overall level of smoothing comparable to the original case where 30 steps with $\Delta t=0.1$, the number of iterations has to be reduced in proportion to the increase in Δt . For $\Delta t=0.25$, the diffusion is about 2.5 times stronger, so roughly $30/2.5$ 12 steps give a similar effect. For $\Delta t=0.4$, the diffusion is 4 times stronger, leading to about $30/4$ 8 steps.

```
1# Testing larger  $\Delta t$  values (0.25 and 0.4)
```

```
## [1] 1
```

```
# New diffusion coefficients
```

```
k_025 <- 0.25
```

```
k_04 <- 0.40
```

```
# Fewer steps because smoothing is much stronger
```

```
smooth_dt025 <- diffuse_matrix(initial_matrix, steps = 12, k_025)
```

```
smooth_dt04 <- diffuse_matrix(initial_matrix, steps = 8, k_04)

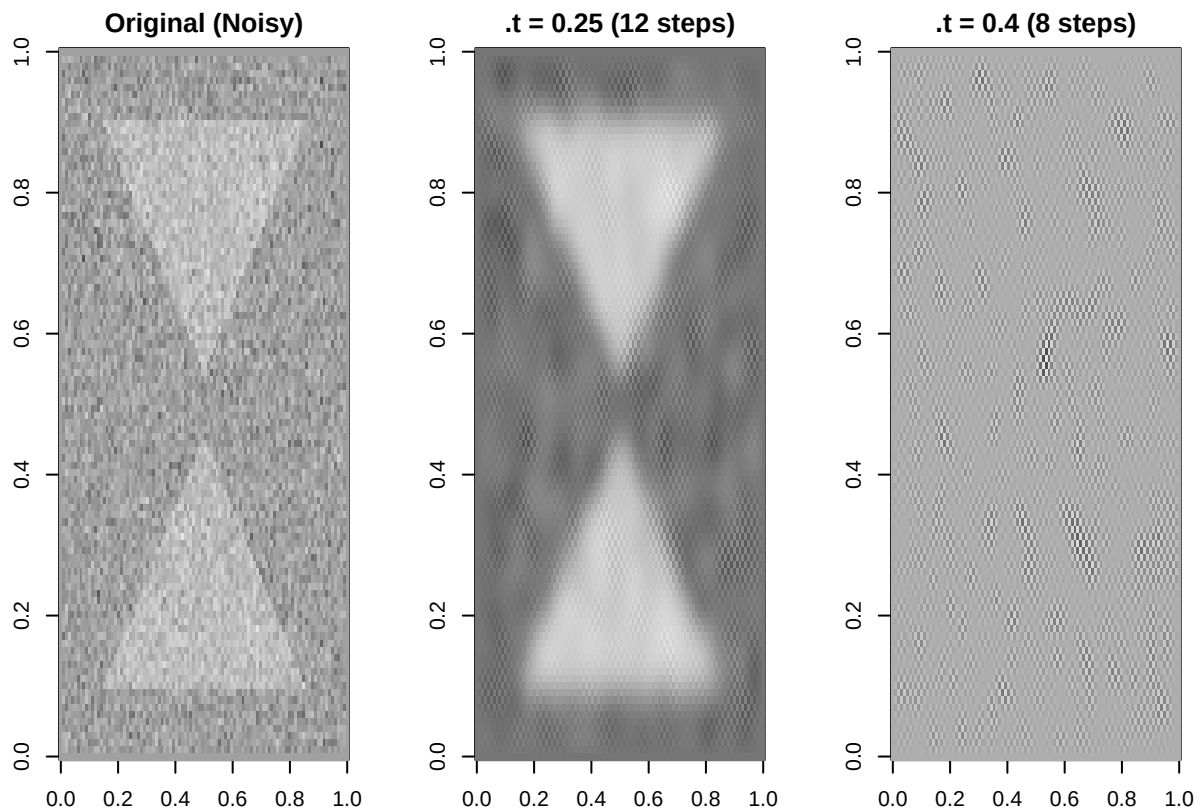
# Saving plot for

par(mfrow=c(1,3))
par(mar=c(4,4,2,1))

image(initial_matrix,
      col = gray.colors(256),
      main = "Original (Noisy)")

image(smooth_dt025,
      col = gray.colors(256),
      main = " $\Delta t = 0.25$  (12 steps)")

image(smooth_dt04,
      col = gray.colors(256),
      main = " $\Delta t = 0.4$  (8 steps)")
```



A larger time step makes the diffusion process stronger in each iteration. With $\Delta t=0.25$, the smoothing happens noticeably faster, and fewer steps are needed to reduce the noise while still keeping the structure recognisable. When Δt is increased to 0.4, the update becomes too strong and crosses the stability limit of the explicit diffusion scheme. This causes numerical artifacts instead of proper smoothing, which appear as oscillating, checkerboard-like patterns in the image. This shows that increasing Δt speeds up denoising only up to a certain point, beyond which the method becomes unstable.

c. The diffusion equation is suitable for image denoising because it suppresses high-frequency variations, which typically correspond to noise, while preserving large-scale structures. By averaging neighbouring pixel values in a controlled way, diffusion reduces random fluctuations that interfere with feature detection in automated image analysis. Cleaner images lead to more reliable segmentation, edge detection, and classification, since algorithms become less affected by noise. The duration of the diffusion process is important because it determines the extent of smoothing. If diffusion is applied for too short a time, a significant amount of noise remains, and the underlying structure is not clearly revealed. If it is applied for too long, the image becomes over smoothed, edges blur, fine details disappear, and meaningful patterns are gradually lost. With a large number of time steps, the solution approaches a uniform intensity field, where almost all structural information is removed. For practical image analysis, an intermediate diffusion duration provides the best balance between noise reduction and preservation of important features.